NDS V6 敏感信息识别系统

研究报告

Zerone01 | 2026

答辩交付材料

摘要

我将 NDS V6 定位为面向数据库敏感信息发现任务的本地化识别系统。系统连接 MySQL 与 PostgreSQL 数据库，对表字段、半结构化内容、编码文本、自然语言文本、图片 BLOB 和数据库对象定义进行扫描，最终输出赛事要求的 upload.csv。

在实现过程中，我没有把问题简化为“写更多正则”，而是先把数据库内容按数据形态分流，再交给不同检测器处理。这样做的好处是结构化字段、JSON、编码串、长文本和图片字段都能保留各自的处理逻辑，后续也方便定位问题和继续优化。

> 正式提交结果生成标准 upload.csv，实际运行时间为 18 分钟，实际得分为 92.78%。

需求与约束

赛题并不是单纯检查数据库连接能力，而是要求在有限时间内从复杂数据库内容中找出敏感信息。敏感值可能直接出现在字段值中，也可能藏在 JSON 键值、Base64/Hex/URL 编码串、备注日志、图片二进制字段，以及存储过程或视图定义中。

多数据库

统一数据库连接层，分别适配 MySQL 与 PostgreSQL。

多数据形态

FormRouter 先判断字段形态，再交给对应 Detector。

限时运行

扫描过程以规则引擎和本地 OCR 为主，避免依赖外部网络服务。

标准输出

所有结果封装为 Candidate，最终统一写入 9 列 CSV。

现场复现

exe 与 config.yaml 分离，现场只需修改配置文件。

可解释

每条结果保留来源字段、数据形态、敏感类型和敏感级别。

总体架构



入口 main.py 初始化数据库连接、结构扫描器、字段路由器、六类检测器和导出管道。检测结果统一封装为 Candidate，再由 ExportPipeline 进行后处理。这个结构让我在调试时可以按阶段定位问题：先看字段是否被路由到正确形态，再看检测器是否召回，最后看导出阶段是否被过滤。

关键源码模块

层次	源码位置	职责
入口编排	main.py	读取配置、组织扫描流程、调度检测器、控制输出。
配置层	config.py / config.yaml	解析外置配置，支持 exe 同目录配置文件。
数据库访问	utils/db_connector.py	封装 MySQL、PostgreSQL 连接和查询。
结构扫描	scanners/structure_scanner.py	读取表结构、字段类型、主键和扫描元数据。
字段路由	scanners/form_router.py	根据字段类型、字段名和内容特征判断数据形态。
规则匹配	matchers/core.py / patterns.py	执行敏感类型正则、字段关键词和校验逻辑。
导出管道	exporters/pipeline.py	串联标准化、去重、噪声过滤、结果增强和 CSV 写出。

六类数据形态处理

形态	源码模块	处理重点
structured	detectors/structured.py	普通字段中的姓名、手机号、身份证、银行卡、地址等直接
semi_structured	detectors/semi_structured.py	递归解析 JSON/JSONB，通过键名与值共同判断敏感类型。
encoded	detectors/encoded.py	Base64、Hex、URL 解码后再执行 JSON/KV 解析和规则匹
unstructured_text	detectors/unstructured.py	从备注、日志、描述等长文本中抽取敏感实体。
binary_blob	detectors/binary_blob.py	对 BLOB/BYTEA 图片执行本地 OCR，再识别 OCR 文本。
db_object	detectors/db_object.py	扫描视图和存储过程定义中的硬编码账号、手机号、密钥等

字段路由策略

FormRouter 的作用是把字段先分到合适的处理通道。数据库字段类型、字段名称、内容片段和二进制特征会共同参与判断。例如 JSON/JSONB 字段进入半结构化检测，BLOB/BYTEA 字段进入图片检测或二进制处理，疑似编码内容进入解码检测，普通文本字段则根据字段语义和内容长度进入结构化或非结构化检测。

- 字段名提供语义线索，例如 phone、id_card、address、password、token 等。
- 字段类型提供形态线索，例如 JSON、TEXT、BLOB、BYTEA、VARCHAR。
- 字段内容提供补充判断，例如是否可解析为 JSON、是否符合 Base64/Hex 编码特征。
- 数据库对象定义独立扫描，避免遗漏视图和存储过程中的硬编码敏感值。

匹配与后处理

SensitiveMatcher 是核心匹配器，规则来源包括敏感类型正则、字段关键词和类型校验函数。身份证、银行卡、统一社会信用代码等类型通过格式或校验码逻辑降低误报。

Normalizer 负责全角半角转换、空白清理、手机号和编号归一化、车牌大写、地址前缀修正；Suppressor 过滤短值、全 0/全 1 值和常见噪声；导出增强逻辑来自多轮运行结果对比，重点处理稳定可验证的召回缺口和误报簇。

迭代过程与工程取舍

系统不是一次性写完规则后直接提交，而是经历了多轮运行、比对和回归检查。每轮我都会同时看四类信号：CSV 行数变化、数据形态覆盖、明显误报簇、完整运行时间。只有在召回提升、误报可控、运行时间仍低于限制时，才把对应策略保留下来。

OCR 是最明显的一次取舍。早期我尝试过精度更高但更重的 OCR 方案，单独图片识别质量更好，但完整库扫描时间会明显超过 30 分钟。后续我改用更轻量的本地 OCR 流程，并通过图片格式判断、文本清理和导出前校验补偿一部分精度损失。最终版本没有追求单点最高精度，而是取整体运行时间、召回范围和交付稳定性的综合最优。

BLOB 字段也经过了多次迭代。最初如果把二进制字段全部打开识别，会带来大量无效结果和运行时间压力；后续我把重点放在可识别图片、字段上下文和稳定敏感类型上，减少低价值二进制扫描。地址、统一社会信用代码等类型则通过多轮结果对比补齐规则，保证新增结果能够被来源和类型解释。

输出格式

系统最终输出 upload.csv。为了保证评分程序可直接读取，导出阶段固定字段顺序，不在文档交付包内额外放置中间 Markdown 或数据库环境文件。

定位信息

db_type、db_name、table_name、field_name、record_id 用于定位来源。

形态信息

data_form 记录 structured、encoded、binary_blob 等数据形态。

分类信息

sensitive_type 与 sensitive_level 标注敏感类型和敏感级别。

原始取值

extracted_value 按赛事要求保留明文。

脱敏与权限控制方案

赛事评分要求输出明文 extracted_value。实际治理落地时，可基于敏感级别选择脱敏策略。L4 类型使用强脱敏或不可逆哈希；L3 类型保留必要业务片段；L2 类型按业务场景降精度或局部隐藏。

- 身份证建议保留前 6 位和后 4 位；银行卡建议保留前 6 位和后 4 位。
- 手机号保留前三后四；地址保留省市区并隐藏门牌；密码密钥不展示原文。
- 扫描程序建议使用只读数据库账号，避免具备写表、删表或修改结构权限。
- upload.csv 含明文敏感值，应限定访问范围；日志不记录数据库密码。

运行结果

18 分钟
实际运行时间

92.78%
实际得分

6 类
覆盖数据形态

系统完成完整扫描，结果文件 upload.csv 已放入最终提交包根目录。系统实际运行时间为 18 分钟，实际得分为 92.78%。

提交 CSV

upload.csv，符合赛事提交格式。

实际运行时间

18 分钟。

边界说明

NDS V6 是规则引擎与本地 OCR 结合的工程系统，不依赖外部在线服务。当前版本围绕比赛数据库规模和输出格式完成调优，迁移到其他真实业务环境时，应重新评估敏感类型范围、字段命名习惯、数据库 schema 和误报容忍度。

- 当前支持 MySQL 与 PostgreSQL，PostgreSQL 扫描范围主要面向 public schema。
- OCR 对低清晰度图片、非图片文件和非常规图片格式存在识别边界。
- 导出增强逻辑来自多轮迭代取最优效果，迁移到新数据集时需要重新回归验证。
- 系统负责识别和导出，不直接修改原始数据库，不替代数据库权限管理系统。

结论

NDS V6 将数据库敏感信息识别拆解为结构扫描、形态路由、专项检测、匹配校验和导出治理五个阶段。该设计避免了单一路径处理复杂数据的局限，使系统能够覆盖普通字段、JSON、编码文本、自然语言文本、图片 BLOB 和数据库对象定义。

从交付角度看，系统已经形成源码、可执行程序、外置配置、标准 CSV、操作说明和答辩材料的完整闭环，能够支撑决赛答辩中的现场说明和复现实验。